Name:	**ID:**
Adrián García Cuéllar	2996024
Roberto Isaac Cabral Sifuentes	2993421
Rafael Alfonso Medellín Guillén	2956234
David Silva Padilla	2958539
Juan Pedro Sánchez Villalobos	2961114
Course: Knowledge Management Systems and Technologies	**Teacher:** Alfredo Paul Aguirre Torres
Date: November 14, 2026	**Activity:** Activity 4

Universidad Tecmilenio

Model Description of the Recommendation Engine

For this recommendation engine, we used the Sentence Transformer model labelled ‘all-MiniLM-L6-v2’. This model allows us to transform the document text into numerical representations called embeddings, which are necessary to capture the semantic meaning of the information provided and allow the recommendation system to do the direct comparison with queries.

In this project, the model is used to generate embedding’s from each document’s title and content. When a user types a query, this one is also converted and compared based on the cosine similarity to return the top 5 most semantically similar documents. It’s important to point out the model is already pretrained and initialized from the start of the execution of the Python script where it was implemented.

Query, Seed Document, or Recommendation Scenario Specifications

Scenario: A text fragment of one of the original documents is provided as a query.

Example Query: “Several users reported that they could not sign in to the client portal after completing the password reset process. The reset page displayed a success message, but the new password was rejected on the login screen” (Text fragment obtained from DOC014 in corpus.csv file from Activity 3).

Objective: Evaluate whether the recommender can identify the original document as one of the most relevant in terms of similarity.

Expected Behavior: If the scenario results as expected, the original document should appear among the top recommendations, as well as other documents which are still semantically relevant to the query information. This will prove the model’s ability to identify semantic relationships without needing to obtain a full document as an input.

Validation Table

The validation for relevant results was done using the query defined on the previous section. The query to identify relevancy focuses on topics like user authentication, password resets, login failures, and account access.

Ground truth classifications were manually established by reviewing the content on all 25 documents in the corpus. Documents were classified as relevant if their contents seemed to fit with the mentioned topics, regardless of their document category. Also, a document was classified as not relevant if it discussed unrelated

website operations, technical incidents, or general procedures without providing much information about the identified access and authentication topics.

The following table shows the ground truth classifications, along with the actual results obtained by sending the query through the recommendation system:

Document	Ground Truth (Relevant/Not Relevant)	Retrieved by the System
DOC001	**Yes**	**No**
DOC002	**No**	**No**
DOC003	**No**	**No**
DOC004	**No**	**No**
DOC005	**No**	**No**
DOC006	**No**	**No**
DOC007	**No**	**No**
DOC008	**No**	**No**
DOC009	**No**	**No**
DOC010	**No**	**No**
DOC011	**No**	**No**
DOC012	**No**	**No**
DOC013	**Yes**	**No**
DOC014	**Yes**	**Yes**
DOC015	**No**	**No**
DOC016	**No**	**Yes**
DOC017	**No**	**No**
DOC018	**No**	**Yes**
DOC019	**No**	**No**

DOC020	**No**	**No**
DOC021	**No**	**No**
DOC022	**Yes**	**Yes**
DOC023	**Yes**	**Yes**
DOC024	**No**	**No**
DOC025	**No**	**No**

Legend:

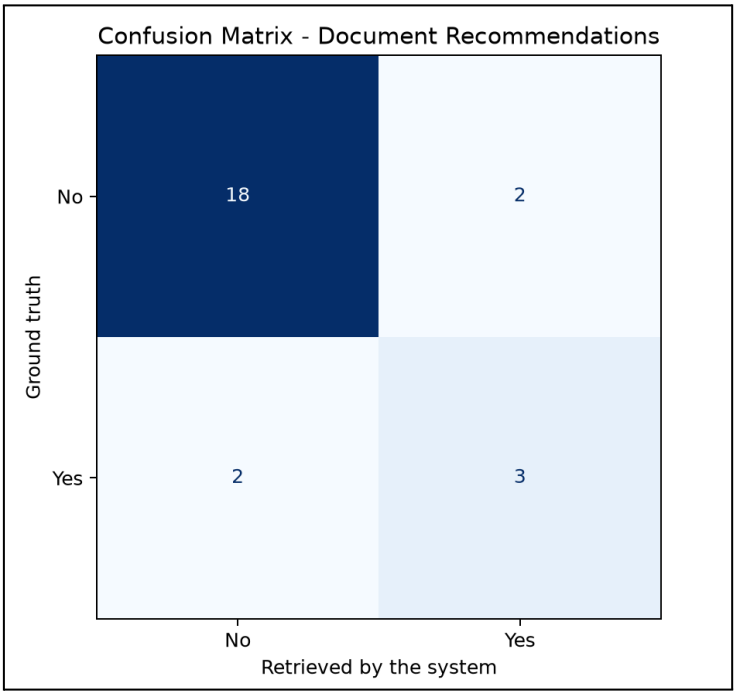
- **True Positive (TP)**: Relevant and retrieved.
- **False Positive (FP)**: Not relevant but retrieved.
- **False Negative (FN)**: Relevant but not retrieved.
- **True Negative (TN)**: Not relevant and not retrieved.

Precision, Recall, and F1-score Metrics

Metric	Result	Interpretation
Precision	60%	Of the 10 files that the model can recommend, only 6 are correct, showing a good balance, with those at the top of the list being the most relevant.
Recall	60%	Of all the files in the database, the model displays 60% of those that are relevant after a query, leaving the other 40% hidden. This is acceptable, as it still shows most of the files, but it can be improved.
F1-Score	60%	Having a value of 60% confirms that the model does not have any bias towards either extreme (recommending little to avoid failure or recommending everything to succeed solely based on volume).

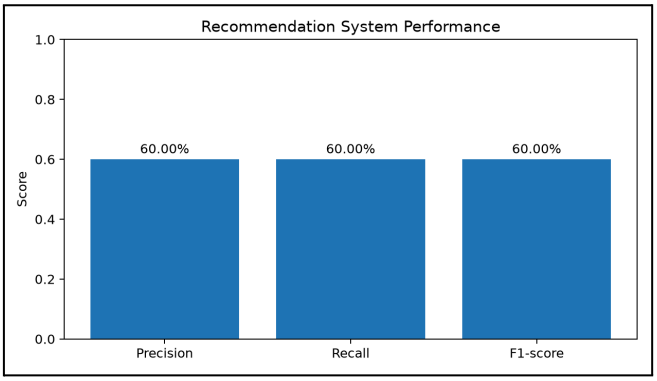
Confusion Matrix Plot

After implementing a Python script to retrieve the same metrics as the ones shown previously, the following Confusion Matrix was generated as well. It represents the four possible classifications (TP, FP, FN, TN) and their frequencies on the scenario:



Data Visualizations of the Performance Results

The following graph was obtained as an output for the scikit-learn library, representing a direct comparison between the three key metrics considered in this activity: Precision, Recall, and F1-score:



Universidad Tecmilenio

Performance Analysis and Breakdown

Our implementation of semantic search using our Python script resulted in a moderate metric performance. While neither precision nor recall was above the other, the results of 60% is just above the 50% in common confidence level but could certainly become higher with some fine tuning.
We limit the returnable amount of documents to five, this is to comply with the requirements of Activity 3. In which case, we cannot confidently determine that the search model returns a lesser amount of relevant documents or a higher amount of irrelevant documents since we use a fixed number.
To assign which documents that the search should return we manually assign a positive score in the Ground Truth table signified with a YES. This is why, even though we do not have a relevant/irrelevant classifier, we are able to get some metrics relevant for the activity.
Of course a manual behaviour is not optimal in an organizational context and it would imply manual work from a dedicated staff member to classify documents by relevancy on each search. Talking about false positives and false negatives, as stated before the model neither recommended few documents to avoid failure nor returned all documents to succeed, which is considered something positive for the organizational context.
This is where implementing Continuous Feedback Loops would benefit our system, since we can obtain relevancy scores by real users. The obtained feedback can be used to get real metrics and determine a course of action to improve our current semantic search model. Another method to improve the system that can be applied now, is the use of document metadata like date, author or url cite, would add more context to each search, making specific searches easier to find.

Reflective Infographic

Universidad Tecmilenio

Organizational Value of Evaluation



How evaluation metrics objectively measure intelligent system quality?

Well-balanced metrics in a system provide the opportunity to better understand the data that both the company and the user are looking for. Quality depends entirely on them; if they are not used correctly, even flawless calculations can lead to misunderstandings or errors.







How they help improve information retrieval?

These metrics make it easier to identify system failures, allowing developers to better fine-tune the models to provide faster and more specific responses.









Why they are vital for decision-making in organizations that rely on knowledge management systems?

Having the right metrics leads to more reliable results for the company, which helps it make more confident decisions and can prevent financial problems caused by inaccurate or incomplete information.

JADIR